

Characterizing software development method using metrics

Doaa M. Shawky^{*†} and Salwa K. Abd-El-Hafiz

Engineering Mathematics Department, Faculty of Engineering, Cairo University, Giza 12613, Egypt

ABSTRACT

This work investigates whether the development methodology affects some metrics that are related to the quality of the generated code. Trying to characterize the development methodology using software metrics could be beneficial to developers who use agile approaches extensively. Therefore, this study sheds light on the metrics that might be highly affected by following agile approaches. The values of these metrics need to be tracked, because they might result in a quality-related issue, such as error-proneness, for example. The proposed approach employs different feature selection and classification methods, which include artificial neural networks. We used 11 object-oriented systems of various sizes and functionalities. Five of the used systems were developed using agile approaches, while the rest were developed using non-agile methodologies. The classes and methods of the used systems were represented by a set of metrics that shows some complexity, cohesion, and coupling features of the systems. The generated metrics are used as features to classify between the two different types of the development methodologies using a feed-forward artificial neural network. The obtained high classification accuracy shows the strong relationship between the used metrics and the type of the followed development process. Copyright © 2015 John Wiley & Sons, Ltd.

Received 8 June 2015; Revised 6 September 2015; Accepted 16 November 2015

KEY WORDS: agile development; classification; feature selection; software metrics; ANN

1. INTRODUCTION

Agile software development approaches are a set of practices that is based mainly on iterative and frequent code changes in response to users' requirements [1]. The term agility refers to rapid movements in different directions [2]. The literature includes many definitions for the term 'agility'. For example, in [3], agility is defined as follows: 'An agile corporation is a fast moving, adaptable and robust business enterprise capable of rapid reconfiguration in response to market opportunities'. This definition relies on adaptability, which is achieved through reconfiguration capability, with processes, structures, organization, and people as the key issues. Applying this definition to a software development process results in an iterative process that promotes close cooperation between the development team and the customers. This usually leads to more adherence to customers' requirements.

On the other hand, software metrics are a set of measures that provide some insights about the developed software [4]. The literature includes many works that employed software metrics for various tasks. In [5], a study was performed to predict the power of an object-oriented (OO) metrics suite that was proposed by Chidamber and Kemerer [6] as quality measures. The authors concluded that many of Chidamber and Kemerer's OO metrics are useful to predict class fault-proneness

^{*}Correspondence to: Doaa M. Shawky, Engineering Mathematics Department, Faculty of Engineering, Cairo University, Giza 12613, Egypt.

[†]E-mail: doaashawky@staff.cu.edu.eg

during the early phases of the life cycle. Moreover, many researchers have used software metrics for fault prediction (e.g., [7]), component classification as fault prone or not (e.g., [8]), effort estimation (e.g., [9]), estimation of software information content (e.g., [10]), clone detection [11–14], and re-engineering or maintenance activities [15, 16].

For instance, in [17], the authors have used several OO complexity metrics to measure the predictive power of such metrics of the quality and reliability of OO software systems. The used metrics include McCabe cyclomatic complexity, weighted methods per class, average method complexity, and four more complexity metrics. The predictive power of these metrics was investigated using statistical methods. Six versions of the Mozilla Rhino system, which has a highly iterative development process that is very similar to agile development, were analyzed. The obtained results proved that OO metrics can predict fault-prone classes in Rhino. The limitation is the analysis of one system only, which makes the generalization of their findings questionable. In [7], a similar study was conducted to analyze the effect of some OO metrics on predicting the faulty classes. The main difference is the inclusion of OO inheritance metrics in the study. The case study included 12 different systems that were developed by undergraduate engineering students. The study concluded that cohesion and coupling metrics are correlated to fault-proneness. The limitation of the presented approach is the small-sized case studies that were developed by non-professional developers. Thus, the results may be biased and non-generalizable.

Another study was conducted on the evolution of a web development project that used software metrics and agile practices [18]. The authors concluded also that there is a relationship between the evolution of the applied metrics and the applied agility practices. Moreover, Capiluppi *et al.* have studied the evolution of a system that was developed using Extreme Programming (XP) [19]. McCabe cyclomatic complexity number was used as a measure of complexity. The authors compared this number between successive releases. The study concluded that agile approaches allow for smooth growth and less complexity. The major threat to validity of the used approach is that they only used one software system in the study. Giblin *et al.* also studied two similar applications that were developed by the same team [20]. While one of these applications was developed using agile practices, the other was developed using the waterfall method. The differences between the two systems were characterized using software metrics. The authors also concluded that agile practices yielded code with better quality and maintainability characteristics.

In addition, in [21], the authors have studied the adoption of agile practices in industry. The study investigated XP almost exclusively. The main findings of the study are that there is a lack of a complete adoption of agile practices and that the number and quality of studies on agile software development need to be increased. Also in [22], the author has studied the impact of agile practices on software quality in a large distributed organization. The study employed defect data metrics and surveys of involved personnel, and revealed the great effect of the adoption of agile practices on the software quality. However, the study used software systems from the same organization, which constitutes an external threat to the obtained results. In [23], the authors used logistic regression to build a model for identifying the development type using a set of metrics as features. They concluded that lack of cohesion, fan in, and depth of inheritance are highly affected by following agile approaches. Moreover, a comparison between the two development types and how they affect the stability of the produced system is presented in [24] by adopting chaos theory. It was concluded that metrics, which characterize subsequent versions of systems that were developed using both agile and non-agile processes, can be modeled using chaos theory. In addition, the metrics characterizing subsequent versions of agile systems were ‘more chaotic’ than those of the systems that were developed using non-agile approaches.

In this paper, we present an approach that studies the effect of the development process on the quality of the produced software systems. The main research question that this work investigates is as follows. Is there a relationship between the type of the development method used and the values of some quality-indicator metrics?

The study includes several large-scale open object-oriented systems with various functionalities. In addition, a large set of product metrics were employed. We start our analysis by creating two lists that contain the set of classes and methods of the used systems. These two lists are then characterized by the set of used metrics for each list. We employed the static analysis tool Understand (www.scitools.com)

for creating the metrics values. In the second step, we applied classification techniques using artificial neural networks (ANN) as an engine. We built two feed-forward ANNs where the set of features are the metrics values, and the dependent variable is the type of development (encoded as 1 and 0 for agile and non-agile developments, respectively). Finally, we calculated the classification accuracy of each constructed ANN to assess their discriminating power. High classification accuracy can be interpreted as the high quality of the used features or metrics for representing the used systems. Consequently, it can be concluded that the type of the development process has a high effect on the attributes that are represented by the set of used metrics.

The rest of the paper is organized as follows. Section 2 introduces a background about the used tools and theories. In Section 3, the proposed approach is presented in detail. Section 4 presents the experimental study. Threats to the validity of the proposed work are discussed in Section 5. Finally, Section 6 presents the conclusions and draws some outlines for the future work.

2. DEFINITION OF RELATED TERMS

In this section, a brief background about some of the used tools and concepts will be introduced.

2.1. Agile software

The definition of agility emphasizes speed and flexibility as the primary attributes of an agile organization [25]. Moreover, in [26], the author states that the effective response to change and uncertainty is an important attribute of agility. In addition, some authors state that responding to change in proper ways and exploiting and taking advantages of changes are the main factors of agility [27]. Also, in [28], the author states that agility is made up of flexibility and speed.

When the basic attributes of agility are applied to the software development processes, this results in an agile software development. Agile software development refers to a group of software development methodologies based on iterative development, where requirements and solutions evolve through collaboration between self-organizing cross-functional teams. Agility term was in use in the year 2001 when the Agile Manifesto was formulated [1].

In general, agile methods promote a project management process that employs frequent inspection and adaptation, encourages teamwork, self-organization and accountability, and a set of engineering practices that allow for rapid delivery of software. Moreover, it aligns development with customer needs and company goals. Some of the well-known agile software development methods include SCRUM, Crystal Clear [29], XP, adaptive software development [30], feature-driven development, and dynamic systems development method [31]. A review on these methods and other agile methods can be found in [21].

2.2. Software metrics

A software metric is a measure that generates a numerical value for a part or a specification of the software [32], [4]. Some common metrics include those that characterize the size of the software (e.g., source lines of code, number of classes, and number of files), the complexity of the software (e.g., cyclomatic complexity and Function Point Analysis), or metrics that measure coupling and cohesion among software entities (e.g., maximum nesting and lack of cohesion percentage). In order to characterize a software using metrics, we must choose a set of metrics with low correlation. In this way, we make sure that the added information is actually used [33]. A review on software metrics can be found in [6].

2.3. Feature selection and classification

Feature selection (FS) is a key step in the data mining process. In FS, the objective is to select the smallest subset of features that reduces complexity and ensures generalization. There is a great deal of research on FS, for example, [34]. FS algorithms can be roughly grouped into two categories: filter methods and wrapper methods. Filter methods rely on general characteristics of the data to evaluate and to select the feature subsets without involving the chosen learning algorithm. They are

usually used as a pre-processing step because they are simple and fast. A widely used filter method is to apply a univariate criterion separately on each feature, assuming that there is no interaction between features. Some common filter methods include t -test, Wilcoxon rank sum, and information gain (IG) using entropy [35].

In the t -test, if we have two classes, we assume that the samples are independent and come from normally distributed populations. We test the null hypothesis that the samples have equal means against the alternative hypothesis that they have different means by calculating the t -statistics using Eqs. (1–3) [36]. Then, if the resulting p -value is larger than the significant level α (usually set to 0.05), we reject the null hypothesis.

$$t = \frac{\bar{x}_1 - \bar{x}_2}{SE(\bar{x}_1 - \bar{x}_2)} \quad (1)$$

$$SE(\bar{x}_1 - \bar{x}_2) = S_p \sqrt{\left(\frac{1}{m} + \frac{1}{n}\right)} \quad (2)$$

$$S_p = \sqrt{\frac{(n-1)S_1^2 + (m-1)S_2^2}{n+m-2}}, \quad (3)$$

where m and n are the number of samples belonging to class 1 and 2, respectively, $\bar{x}_1$ and $\bar{x}_2$ are the means of the samples of the two classes, and S_1 and S_2 are the standard deviations for the two classes.

In Wilcoxon rank-sum method, we start by combining the $m+n$ observations of the two classes, then we rank the combined set of observations. The rank of X_i among the N observations, denoted $R(X_i)$, equals the number of X_j $s \leq x_i$, where x_i is the value of observation i . The test statistics is the sum of the ranks (W) of the observations for class 1 (or class 2). The null hypothesis in this case is that the distributions of the two classes are the same. Equation (4) is used to calculate the p -value [37].

$$p = \frac{\# \text{ of rank sums } \leq W}{\binom{m+n}{n}} \quad (4)$$

Another commonly used filter method is the IG method. In this method, features are selected based on the IG, which is calculated using Eqs. (5)–(6).

$$IG(S_x, x_i) = H(S_x) - \sum_{v=\text{values}(x_i)} \frac{|S_{xi=v}|}{|S_x|} H(S_{xi=v}) \quad (5)$$

$$H(S) = -p_+(S) \log_2 p_+ - p_-(S) \log_2 p_-, \quad (6)$$

where S_x is the set of training samples, x_i is the vector of the i th variable, $S_{xi=v}$ is the set with i th variable having value v , H is the entropy, and $p_{\pm}(S)$ is the probability of a training sample to be of the first or second class. Continuous features are usually discretized using information theoretic binning [38].

On the other hand, wrapper methods use the performance of a chosen learning algorithm to evaluate each candidate feature subset. Examples of known wrapper methods include forward, backward, and random FS [39]. In these methods, a newly generated pool of features is used to build a classifier that is learnt to classify the data based on the set of used features. The classification results are used to assess the discriminating power of the used features. It should be mentioned that classification results depend not only on the FS method used, but also on the employed learning algorithm. Some

of the common classification algorithms include logic-based classifiers such as decision trees [40], perceptron-based classifiers such as ANNs [41] and statistical-based classifiers such as Linear Discriminant Analysis [42], and k-Nearest Neighbor [43]. In [35], an extensive review of existing work on FS is provided.

Artificial neural networks are commonly used as simple and efficient data analysis tools, which are known for their good learning and generalization abilities [44]. They have been used before in several pattern recognition applications and have proved their efficiency. In ANNs-based classification approaches, the dataset is commonly divided into two subsets; the training and the test dataset. The test set is used to provide performance measures of the trained classifier. The training is stopped when generalization can no more be improved because of over-fitting. A commonly used ANN includes the multi-layer feed-forward ANN. In addition, the supervised learning (classification) is usually implemented with a Levenberg–Marquardt back-propagation algorithm [45]. This type of ANNs is known for its fast and good generalization [46]. In addition, in this type of ANNs, the back-propagation algorithm is widely used. This training procedure adjusts the connection weights of a multilayer perceptron (MLP) [47, 48]. The Levenberg–Marquardt algorithm is a least-squares estimation algorithm based on the maximum neighborhood idea. In the maximum neighborhood, the subset of data used for the weighted least squares is determined by a nearest neighbor algorithm. A MLP consists of three layers: an input layer, an output layer, and one or more hidden layers. Figure 1 shows a general structure of an MLP with one hidden layer.

Each layer is composed of a predefined number of neurons. The neurons in the input layer only act as buffers for distributing the input signals x_i to neurons in the hidden layer. Each neuron j in the hidden layer sums up its input signals x_i after weighing them with the strengths of the respective connections w_{ij} from the input layer, and computes its output y_j as a function f of the sum as given by (7).

$$y_j = f\left(\sum w_{ij}x_i\right), \quad (7)$$

where f is the activation function that is necessary to transform the weighted sum of all signals incoming to a neuron. The activation function f can be a simple threshold function, a sigmoidal, hyperbolic tangent, or radial basis [46]. The output of neurons in the output layer is similarly computed. In addition, during the training phase, weights are updated according to (8) and (9).

$$w_{ij}(t+1) = w_{ij}(t) + \Delta w_{ij}(t), \quad (8)$$

$$\Delta w_{ij} = [J^T(w)J(w) + \mu I]^{-1} J^T(w)E(w), \quad (9)$$

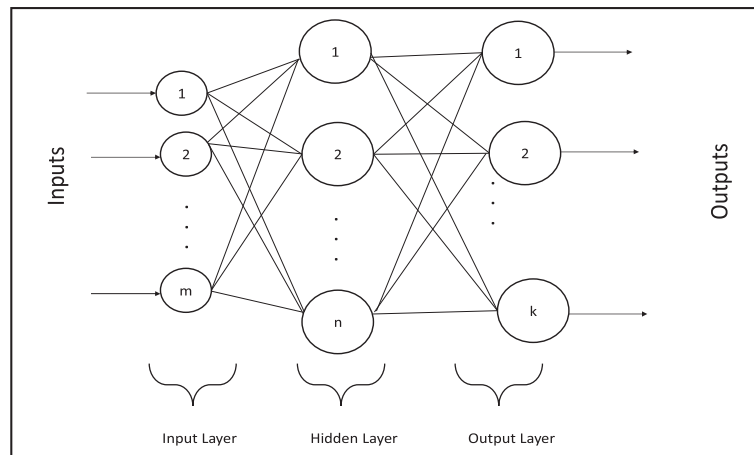


Figure 1. The general structure of a multilayer perceptron.

where J is the Jacobian matrix, μ is a constant that represents the learning rate, I is the identity matrix, and $E(w)$ is an error function.

The performance of the ANN can be assessed using the confusion matrix of the output. For a two-class classification problem, the confusion matrix is typically similar to the one presented in Table I. The variables TP, FP, FN, and TN represent the number of true positives, false positives, false negatives, and true negatives, respectively. Usually, the common performance measures, which can be calculated out of the confusion matrix, are the average classification accuracy, the sensitivity, and the specificity of the classifier. The classification accuracy for class 1 is equal to $TP/(TP + FP)$. On the other hand, the classification accuracy for class 2 is equal to $TN/(FN + TN)$. Hence, the average classification accuracy is equal to the average of these two values. In addition, the sensitivity (recall) is equal to $TP/(TP + FN)$. Meanwhile, the specificity is equal to $TN/(FP + TN)$.

2.4. Stratified sampling

Stratified random sampling is a population sampling method where the population is divided into mutually exclusive subgroups, called strata, and random sampling is then applied to each stratum [49]. The common stratified sampling approaches include proportional stratified sample and disproportional stratified sample. In a proportional stratified sample, the size of each stratum in the sample is proportional to the size of the stratum in the population. Whereas, in a disproportional stratified sample, the size of each stratum is not proportional to its size in the population.

Usually, stratified sampling has high statistical precision. In addition, it yields a small sample size, which can save the analysis time without compromising accuracy [50]. Furthermore, this method can be used to obtain a balanced sample to make sure that the results of the analysis will not be biased by the stratum that has a large number of data points.

3. PROPOSED APPROACH

The proposed approach is presented in Figure 2. As shown in the figure, it consists of the following main steps, which will be presented in detail in the following subsections:

- Creating metrics values for the used systems.
- Creating two separate lists: a class-metrics and a method-metrics lists.
- Building two ANNs and training them to classify between the two differently developed classes and methods.
- Assessment of the performance of the two ANNs.

The analyzed systems consist of 11 case studies with varying sizes and functionalities. Among the 11 used systems, five of them were developed using agile methods, and the rest were developed using the traditional waterfall method. These systems were obtained by applying an internet search using Google search engine with the keywords 'agile development+source code'. By investigating the documentations related to the resulting systems, we kept only those systems in which there is an explicit reference to the adoption of agile approaches during development. For those systems with

Table I. A typical confusion matrix for a two-class classification problem.

Output/actual class		Actual (target) class		
		Class 1	Class 2	
Output class	Class 1	•TP	•FP	• $TP/(TP + FP) = C1$
		• $TP/\# \text{ of samples}$	• $FP/\# \text{ of samples}$	• $FP/(TP + FP) = C2$
	Class 2	•FN	•TN	• $TN/(FN + TN) = C3$
		• $FN/\# \text{ of samples}$	• $TN/\# \text{ of samples}$	• $FN/(FN + TN) = C4$
		• $TP/(TP + FN) = \text{Recall}$	• $TN/(TN + FP) = \text{Specificity}$	•Average (C1, C3)
		• $FN/(TP + FN)$	• $FP/(TN + FP)$	•Average (C2, C4)

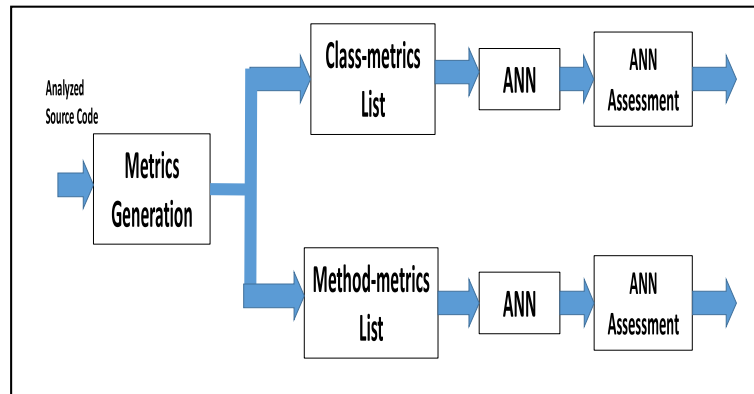


Figure 2. The proposed approach. ANN, artificial neural network.

no such documentations, we checked the developers' forums to make sure that agile practices were applied. Thus, we filtered out those systems with no evidence of the adoption of agile approaches. Consequently, we found a limited number of systems, which were developed using agile methods, with their source code available for download. A description of the used systems is provided in Table II. In addition, a decision attribute is added to indicate whether the corresponding system has been developed using agile approaches or not. This attribute is considered as the dependent variable.

We employed two different sets of metrics: a set that measures some class attributes, in addition to another set for measuring methods' properties. Table III shows the used class-metrics; meanwhile, Table IV presents the method's metrics. The first columns in the tables indicate the symbol that will be used when the corresponding metric is being referred to throughout the paper. Meanwhile, the name and the meaning of each metric as given in the used tool's manual (<http://www.scitools.com/documents/metricsList.php>) are presented in the second and third columns, respectively. Most of the used metrics were previously used in the related work. For instance, CM2, CM3, CM6, CM7, and CM8 are among the used class-metrics. These metrics belong to Chidamber and Kemerer metrics suite [6], which was extensively used in the related work. Also, MM1, MM2, and MM5 have been previously used in [23]. The other used metrics were generated by the analysis tool.

Table II. Used systems.

System, language	Available at:	Number of classes	Number of files	Number of functions	Number of lines (KLOC)	Agile
Eclipse SDK 4.3, Java	http://qualitascorp.com/	33,874	201,570	608,310	2,442	1
Ace, C++	http://download.dre.vanderbilt.edu/	7666	13,975	66,135	2290	1
VTK, C++	http://www.vtk.org/VTK/resources/	3322	3340	7444	2939	1
ITK 4.5.0, C++	http://sourceforge.net/projects/itk/files/itk/4.5/ InsightToolkit-4.5.0.zip/download	3200	3420	7390	2834	1
Suneido, C++	http://sourceforge.net/projects/suneido/files/ Releases/	290	375	3964	968	1
Firefox, C++	http://ftp.mozilla.org/pub/mozilla.org/firefox/ releases/	7823	29,642	159,711	6242	0
Azureous, Java	http://qualitascorp.com/	7660	5038	50196	485	0
Sharp Develop 4.2, C#	http://sharpdevelop.codeplex.com/releases/ view/87331	7012	2371	94,530	37,000	0
VLC, C++	http://www.videolan.org/vlc/download- sources.html	5917	15,186	125,812	4663	0
Flex 4.0, C++	http://sourceforge.net/adobe/flexsdk/wiki/Get %20Source%20Code/	2763	3347	8529	245	0
HOX Chess, C++	https://code.google.com/p/hoxchess/	842	2092	9493	402	0

Table III. Classes' metrics (CM).

Metric's symbol	Metric's name	Metric's meaning
CM1	Count class base	Number of immediate base classes
CM2	Count class coupled	Number of other classes coupled to this class
CM3	Count class derived	Number of immediate subclasses this class has
CM4	Count declarative class method	Number of local methods this class has
CM5	Count declarative class variable	Number of instance variables this class has
CM6	Max complexity	Maximum complexity within class
CM7	Max inheritance tree	Maximum depth of the class in the inheritance tree
CM8	Percent lack of cohesion	100% minus the average cohesion for class data members
CM9	Count line code declarative	Number of declarative lines of code within a class
CM10	Count line codeExe	Number of executable lines of code within a class
CM11	Count line inactive	Number of inactive lines of code within a class
CM12	Count line preprocessor	Number of preprocessors within a class
CM13	Count statement	Number of statements a class has

Table IV. Methods' metrics (MM).

Metric's symbol	Metric's name	Metric's meaning
MM1	CountInput	Number of called subprograms plus global variables set
MM2	CountOutput	Number of calling subprograms plus global variables read
MM3	CountPath	Number of possible paths, not counting abnormal exits or gotos
MM4	Cyclomatic	The average number of independent paths through the functions
MM5	MaxNesting	Maximum nesting level of control constructs (if, while, for, switch, etc.) in the function
MM6	CountLineCode	Number of lines in the function that contain any code
MM7	CountStmt	Total number of statements in the function

The used metrics constitute the set of descriptors (independent variables) that represent each system. We choose a set of metrics that describes various characteristics of a software system in order to be able to reveal the influence of the agility of the development process on these characteristics. We used this set of metrics as we postulate that these metrics may be related to agility to some extent. For instance, it is logical that a rapid delivery of software in agile approaches cannot be easily performed if the software is too complex. This implies possible relationship between complexity metrics and the degree of the agility process.

4. AN EXPERIMENTAL STUDY

In this section, the results of the experimental analysis will be presented. It should be mentioned that although the tool generates a large number of metrics, however, not all metrics were available for languages of the used systems. Thus, we used only the set of metrics, which were calculated across the languages of the used systems. The numbers of entities in the used sets are 11,081 and 176,176 for the class-metrics and method-metrics lists, respectively.

We used ANNs as an engine for our analysis. We implemented two ANNs for the classification of the used lists based on the development type. The two implemented networks employ multi-layer feed-forward back propagation with one hidden layer of 20 neurons. The default number of neurons in the hidden layer of the used network was set to 20, which gives acceptable results in our classification problem. The multi-layer feed-forward back propagation networks are trained using the training set that contains two thirds of the total dataset.

We have started our analysis by using the set of class-metrics, which includes 13 different metrics. The average classification accuracy, recall, and specificity were only 81.5%, 79.8%, and 84.2%, respectively, for the whole set of used metrics as shown in Figure 3. In order to increase the

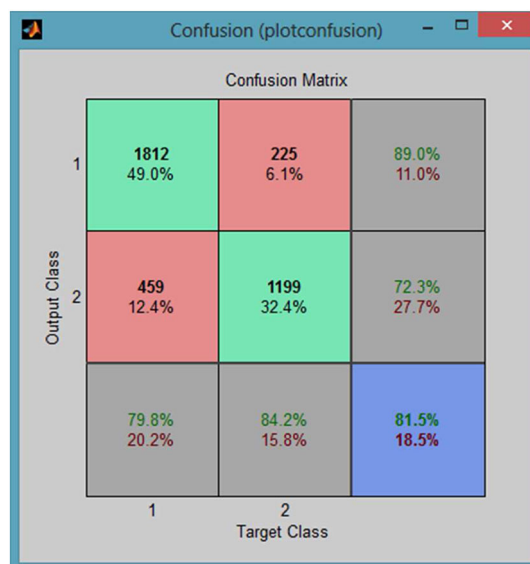


Figure 3. Confusion matrix for the class-metrics set with 13 metrics.

classification accuracy, we applied FS and ranking methods. The used methods include the t -test, Wilcoxon, and IG. The rankings of the features from best to worst are shown in Table V. As shown in the table, the three used methods for feature ranking yielded consistent results for the metrics CM4, CM5, CM7, CM8, CM11, and CM13. In addition, we added CM2 and CM6 as they have p -values < 0.05 according to the t -test method.

It should be mentioned that Wilcoxon rank-sum is similar to the t -test. However, t -test assumes that the samples come from normal distributions; meanwhile, Wilcoxon method does not assume that. In addition, in Wilcoxon rank-sum, equal medians of samples are tested instead of equal means. That is why the rankings of metrics using the two methods are very similar as shown in Table V. Using the IG method, on the other hand, the amount of added information is gradually tested when metrics are added to the list of discriminating metrics. In this case, low IG is obtained if the added metrics are correlated with a previously added metric. Because CM4 is expected to be correlated with CM9 and CM10, the latter two metrics should receive low rankings, which is shown in Table V.

Figure 4 shows the confusion matrix when only the eight metrics are used. As shown in the figure, classification accuracy, recall, and specificity have been enhanced when FS methods are applied. Furthermore, the confusion matrix when we applied proportional stratified sampling with 8000 samples is presented in Figure 5, which also shows a slight enhancement in the classification accuracy, recall, and specificity. It should be mentioned that disproportional stratified sampling gives approximately the same results.

Similar results were obtained when we analyzed the method-metrics list. However, when we applied feature ranking, no consistent results were obtained. Because the p -values of all used features are less than 0.05, we did not remove any of the used metrics in the method-metrics list.

Figures 6 and 7 present the confusion matrix for the method-metrics list when we used the whole set of method-metrics and the stratified set with 20,000 samples, respectively. As shown in Figure 6, the average classification accuracy, recall, and specificity are 96.1%, 96.1%, and 95.9%, respectively.

Table V. Rankings of used features (metrics).

t -test	4	5	7	8	11	13	2	6	10	1	3	9	12
IG	4	5	7	8	11	13	12	9	2	6	10	1	3
Wilcoxon	4	5	7	8	11	13	6	2	10	3	1	9	12

IG, information gain.

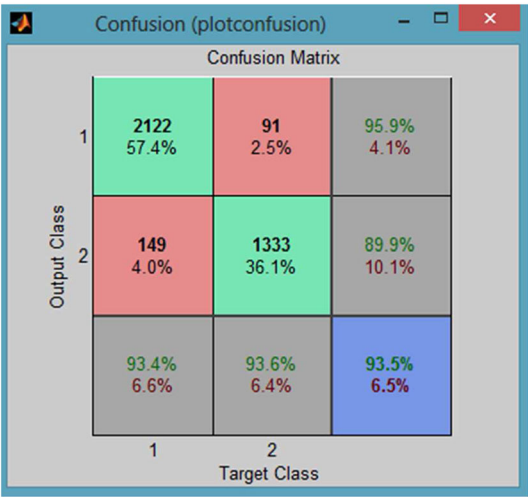


Figure 4. Confusion matrix for the class-metrics set with eight metrics.

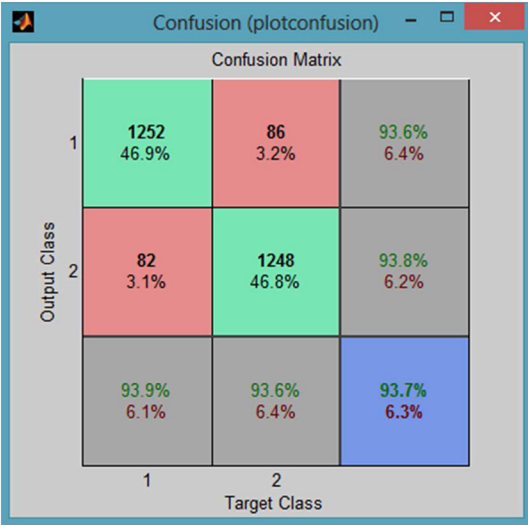


Figure 5. Confusion matrix for the stratified class-metrics set with eight metrics.

Moreover, these values were slightly enhanced to reach 97.1%, 97.1%, and 97.0%, respectively, when stratified sampling was used as shown in Figure 7.

The obtained good classification accuracies between agile and non-agile systems indicate that the used metrics are good discriminators between the two development methods. This means that following agile approaches highly affects attributes of the systems that are quantified by the used metrics.

5. THREATS TO VALIDITY

This work is based on 11 open source systems, which might raise some concerns about threats to its validity that we discuss in this section.

The first issue is that the used systems were identified as being developed using agile or non-agile methods based on internet search, followed by manual investigation of the documentations and/or the systems’ forums. However, it should be mentioned that the degree of agility used and the type of agile approaches that were actually applied in developing the included systems were not declared. In addition, if there was no clear reference to the adoption of an agile approach in the documentation

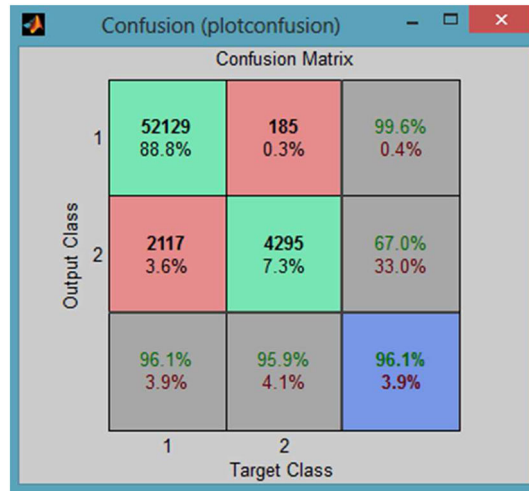


Figure 6. Confusion matrix for the method-metrics set.

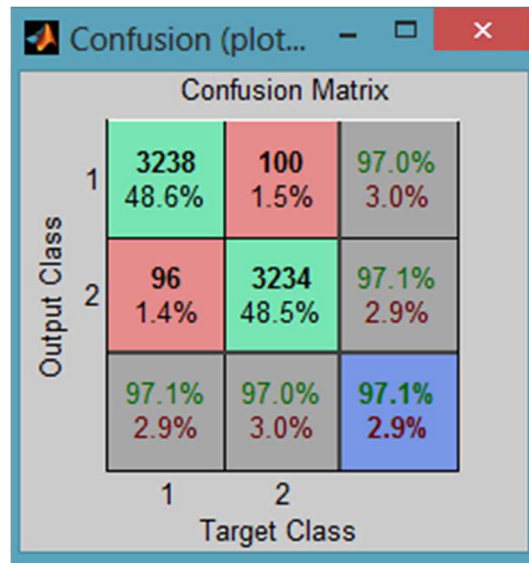


Figure 7. Confusion matrix for the stratified method-metrics.

and/or the system's forums, we assumed that it was developed using purely non-agile approaches. Sometimes, we marked the system as being agile, if we found some related work that treats it so, for example, Eclipse was used in [51] as a case study that was developed using agile approaches.

Another threat to the validity of the presented results may be imposed by the limited number of the used systems, which might add some limitations on the generalization of the findings. Although we used the components of the systems (i.e., classes and methods) as the units of analysis to overcome the problem of limited-size samples, the fact that some of these components belong to the same systems might raise some concerns about the dependence of the samples, which might also affect the generalization of the results.

6. DISCUSSIONS AND CONCLUSIONS

Agile development methods focus on customer satisfaction to measure the quality of the produced software systems. However, customers usually tend to ask for classical solutions that proved their efficiencies before in old problems. These classical solutions do not necessarily work in new

domains. Furthermore, these biased suggestions may raise questions about the quality of the final produced system. Thus, we need to study whether there is an impact of following agile approaches on software quality attributes.

In this paper, we investigated the effect of the development type on some product metrics, which were previously proven to be related to software quality attributes. The results show that the development methodology can be characterized by a set of software metrics. This can be shown by the high

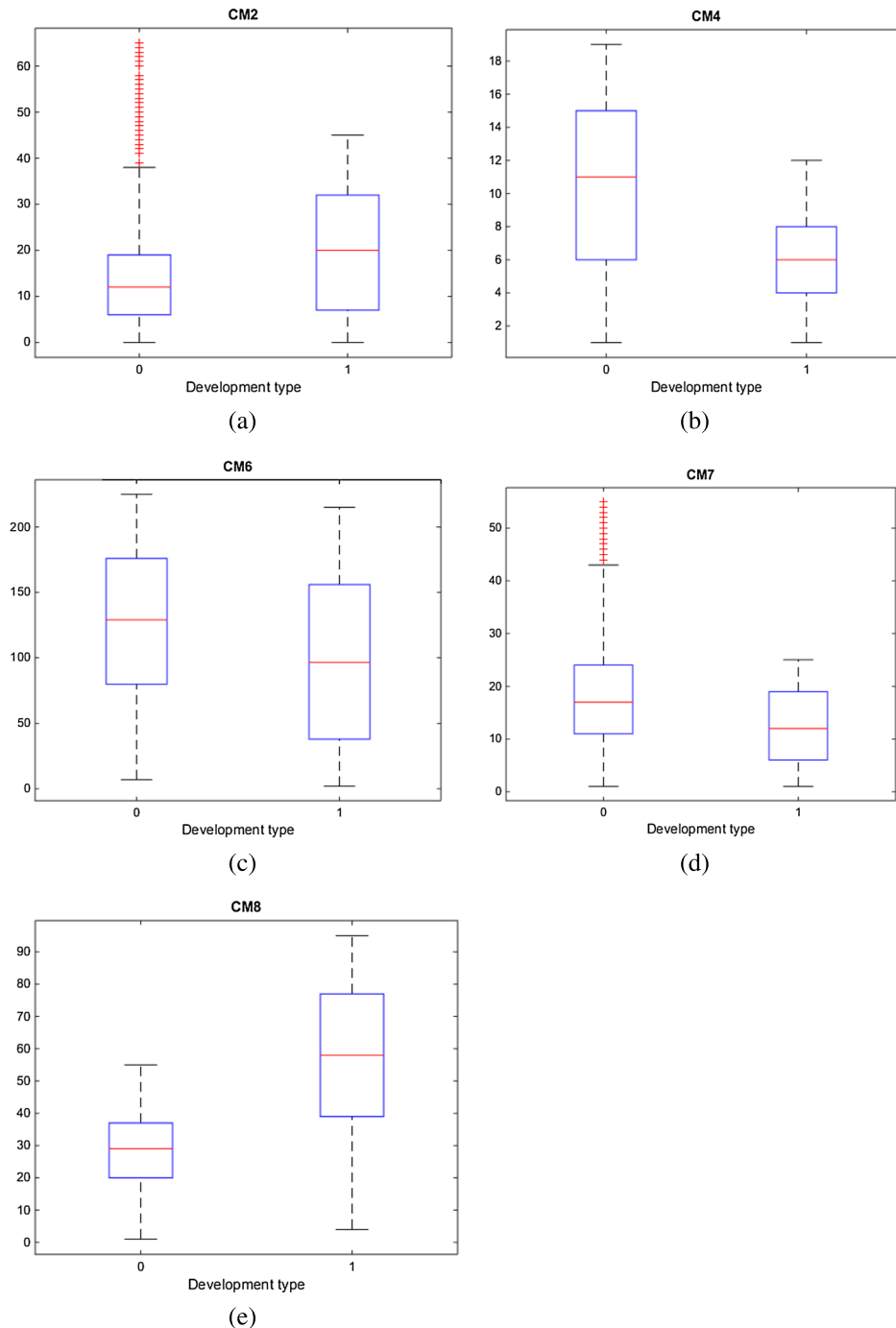


Figure 8. Some class-metrics in used systems grouped by the development type. (a) CM2 (CountClassCoupled) (b) CM4 (CountDeclClassMethod) (c) CM6 (MaxComplexity) (d) CM7 (MaxInheritanceTree), and (e) CM8 (PercentLackOfCohesion). CM, classes' metrics.

classification accuracy of an ANN that is learned to classify between agile and traditional development methodologies using a set of software metrics as features. These metrics need to be tracked when following agile approaches because they might raise some concerns about fault-related issues. In addition, the used set of metrics includes some complexity, coupling, and cohesion metrics. Because coupling and cohesion can be used, in an indirect sense, to measure complexity, therefore, we can conclude that following agile approaches can highly affect the complexity of the produced systems.

An interesting point that needs further investigation is whether agile approaches have a positive or negative effect on the used set of metrics and whether we can modify agile practices to ensure a positive effect. A study was conducted in [17] and [52] to investigate the correlation between defects in a class and a set of OO metrics in several versions of a highly iterative agile system. CM2, CM4, CM6, CM7, and CM8 were among the used metrics. The study concluded the positive correlation between these metrics and the number of defects in a class. In addition, CM4 and CM8 were the most consistent predictors of class quality in all built models. Also, in [23], presented results showed that CM7 and CM8 are highly affected by the development type being agile or not.

Figure 8 shows the boxplots of CM2, CM4, CM6, CM7, and CM8 for the used systems grouped by the development methodology, where 0 denotes systems developed using traditional methods and 1 denotes agile systems. As shown in the figure, CM2 and CM8 have larger values in agile systems in comparison with traditionally developed systems. On the other hand, CM4, CM6, and CM7 have smaller values in agile systems. Several studies showed contradicting evidence regarding the effect of following agile approaches on the quality attributes. For instance, in [20], the authors concluded that following agile approaches has led to the increased values of CM4. On the other hand, CM6 has been decreased, which agrees with our finding. Also in [53], the authors studied the effect of the development method on a set of software metrics including CM4, CM7, and CM8. They concluded that the studied metrics follow nearly the same distributions in the studied agile and non-agile systems. This reveals that the effect of agile approaches on the quality attributes of the produced systems is still an open research question that needs further investigation.

Thus, we can conclude that more attention should be paid to the values of the characterizing metrics when agile approaches are followed, especially for those that might have negative effect on the quality attributes.

As a future work, we intend to study the effect of the metrics that characterize the development method on the quality attributes of the produced systems. Especially those metrics that were not included in previous studies (the methods' metrics). In addition, it would be interesting to understand the reason behind the inclusion of the used metrics as good discriminators between development methods. However, this would not be possible without the availability of further information about the used agile approaches.

REFERENCES

1. Larman C. *Agile and Iterative Development: A Manager's Guide*: Addison-Wesley Professional, 2003.
2. Lee JC, Scott McCrickard D, Stevens KT. Examining the Foundations of Agile Usability with eXtreme Scenario-Based Design 3–10.
3. Kidd PT. *Agile Corporations: Business Enterprises in the 21st Century – An Executive Guide*. Cheshire Henbury, 1995.
4. Cem Kaner SM, Bond Walter P. *Software Engineering Metrics: What Do They Measure and How Do We Know?.*
5. Basili VR, Briand LC, Melo WL. A validation of object-oriented design metrics as quality indicators. *Ieee Transactions on Software Engineering* Oct, 1996; **22**(10):751–761.
6. Chidamber SR, Kemerer CF. A metrics suite for object oriented design. *Software Engineering, IEEE Transactions on* 1994; **20**(6):476–493.
7. Aggarwal K *et al.* Empirical analysis for investigating the effect of object-oriented metrics on fault proneness: a replicated case study. *Software Process: Improvement and Practice* 2009; **14**(1):39–62.
8. Nagappan N, Ball T, Zeller A. Mining Metrics to Predict Component Failures, 2006; 452–461.
9. Jeffery R, Ruhe M, Wiczorek I. Using Public Domain Metrics to Estimate Software Development Effort, 2001; 16–27.
10. Abd-El-Hafiz SK. Entropies as Measures of Software Information. in *Proceedings of the IEEE International Conference on Software Maintenance (ICSM'01)*, 2001 110.
11. Abd-El-Hafiz SK. Efficient Detection of Function Clones in Software Systems using the Fractal Dimension and Metrics, 2011.
12. Abd-El-Hafiz SK. A Metrics-based Data Mining Approach for Software Clone Detection, 2012; 35–41.
13. Shawky DM, Ali AF. An Approach for Assessing Similarity Metrics Used in Metric-based Clone Detection Techniques, 2010; 580–584.

14. Shawky DM, Ali AF. Modeling Clones Evolution in Open Source Systems Through Chaos Theory, 2010; V1-159-V1-164.
15. Kitchenham B. What's up with software metrics? – A preliminary mapping study. *Journal of Systems and Software* 1//, 2010; **83**(1):37–51.
16. Shawky DM. Towards locating features using digital signal processing techniques. *Journal of Engineering and Applied Science* 2008; **50**(1):1–20.
17. Olague HM *et al.* Empirical validation of three software metrics suites to predict fault-proneness of object-oriented classes developed using highly iterative or agile software development processes. *Software Engineering, IEEE Transactions on* 2007; **33**(6):402–419.
18. Concas G *et al.* An empirical study of software metrics for assessing the phases of an agile project. *International Journal of Software Engineering and Knowledge Engineering* Jun, 2012**22**(4):525–548.
19. Capiluppi A *et al.* An Empirical Study of the Evolution of an Agile-developed Software System. 2007; 511–518.
20. Giblin M, Brennan P, Exton C. Introducing agile methods in a large software development team: the impact on the code. *Agile Processes in Software Engineering and Extreme Programming* 2010; **48**:58–72.
21. Dybå T, Dingsøyr T. Empirical studies of agile software development: a systematic review. *Information and Software Technology* 8//, 2008; **50**(9–10):833–859.
22. Korhonen K. Evaluating the impact of an agile transformation: a longitudinal case study in a distributed context. *Software Quality Journal* Dec, 2013; **21**(4):599–624.
23. Shawky D, Abd-El-Hafiz S. The Impact of Agile Approaches on Software Quality Attributes – An Empirical Study. in the 9th International Conference on Software Paradigm Trends, Austria, 2014 49–57.
24. Shawky D. Traditional vs Agile Development – A Comparison Using Chaos Theory. in the 9th International Conference on Software Paradigm Trends, Austria, 2014 109–114.
25. Gunasekaran A. Agile manufacturing: a framework for research and development. *International Journal of Production Economics* 5/20/, 1999; **62**(1–2):87–105.
26. Goldman SL, Nagel RN. Management, technology and agility: the emergence of a new era in manufacturing. *International Journal of Technology Management* 1993; **8**(1–2):18–38.
27. Sharifi H, Zhang Z. Agile manufacturing in practice – application of a methodology. *International Journal of Operations & Production Management*, 01/05/2001, 2001.
28. Lindberg P. Strategic manufacturing management: a proactive approach. *International Journal of Operations & Production Management*. 31/12/1990, 1990.
29. Cockburn A. Crystal Clear: A Human-Powered Methodology for Small Teams: A Human-Powered Methodology for Small Teams: Addison-Wesley Professional, 2004.
30. Dirk R. A Comparison of the Value Systems of Adaptive Software Development and Extreme Programming: How Methodologies May Learn from Each Other,” in *Proceedings of the First International Conference on Extreme Programming and Flexible Processes in Software Engineering*, 2000, 35–50.
31. Keith R. Agile Project Management: Running PRINCE2 Projects with DSDM Atern: 1st impression, 2007.
32. Mills E. Software Metrics, CMU/SEI-88-CM-012, 1988.
33. Te-Shun C *et al.* Correlation-based Feature Selection for Intrusion Detection Design, 2007; 1–7.
34. Hall MA, Holmes G. “Benchmarking attribute selection techniques for discrete class data mining. *IEEE Transactions On Knowledge And Data Engineering* **15**(2003):1437–1447, 2010-04-18T02:00:32–00:00, 2010.
35. Saeys Y, Inza I, Larrañaga P. A review of feature selection techniques in bioinformatics. 2007-10-01, 2007.
36. Ruxton GD. The unequal variance *t*-test is an underused alternative to Student's *t*-test and the Mann–Whitney *U* test. *Behavioral Ecology* 2006; **17**(4):688–690.
37. Rosner B, Glynn RJ. Power and sample size estimation for the Wilcoxon rank sum test with application to comparisons of C statistics from alternative prediction models. *Biometrics* 2009; **65**(1):188–197.
38. Irani KB. Multi-Interval Discretization of Continuous-Valued Attributes for Classification Learning 1993.
39. Kira K, Rendell LA. A Practical Approach to Feature Selection, 1992; 249–256.
40. Kotsiantis SB. Supervised Machine Learning: A Review of Classification Techniques, 2007; 3–24.
41. Rumelhart DE, Hinton GE, Williams RJ. Learning Internal Representations by Error Propagation, 1985; 318–362.
42. Friedman JH. Regularized discriminant analysis. *Journal of the American Statistical Association* 1989; **84**(405):165–175, 1989/03/01.
43. Cover T, Hart P. Nearest neighbor pattern classification. *Information Theory, IEEE Transactions on* 1967; **13**(1):21–27.
44. Basheer IA, Hajmeer M. Artificial neural networks: fundamentals, computing, design, and application. *Journal of Microbiological Methods* 2000; **43**(1):3–31, 12/1/.
45. Moré JJ. *The Levenberg-Marquardt algorithm: implementation and theory. Numerical analysis* 1978; 105–116: Springer.
46. Xin Y. Evolving artificial neural networks. *Proceedings of the IEEE* 1999; **87**(9):1423–1447.
47. Rumelhart DE, Hilton GE, Williams RJ. Learning representations by back-propagating errors. *Nature* 1986; **323**:533–536.
48. Shawky DM, Ali AF. A feature selection method using misclassified patterns. *International Journal of Computer Theory and Engineering* 2011; **3**(5):643–651.
49. Imbens GW, Lancaster T. Efficient estimation and stratified sampling. *Journal of Econometrics* 1996; **74**(2): 289–318.
50. Stehman S. Estimating the kappa coefficient and its variance under stratified random sampling. *Photogrammetric Engineering and Remote Sensing* 1996; **62**(4):401–407.

51. Murgia A *et al.* Empirical Study of Software Quality Evolution in Open Source Projects Using Agile Practices, 2014; 11.
52. Olague HM *et al.* An empirical validation of object-oriented class complexity metrics and their ability to predict error-prone classes in highly iterative, or agile, software: a case study. *Journal of Software Maintenance and Evolution: Research and Practice* 2008; **20**(3):171–197.
53. Destefanis G *et al.* Software Metrics in Agile Software: An Empirical Study. *Agile Processes in Software Engineering and Extreme Programming*, 157–170: Springer, 2014.